애플리케이션 테스트 수행

Fullstack 풀스택 개발자 과정

1조: 박한결, 박영준, 원병선

CONTENTS

1

구동 확인



2

BookControllerTest

원병선



3

BookRepositoryTest
LibraryIntegrationTest

박한결



4

LendServiceTest

박영준



1. 구동 확인 – 테스트



수정 전 일부 실패 → 수정 후 전체 통과

1. 구동 확인 – COMMIT

```
*   23719d1 (HEAD -> main, origin/main, origin/HEAD) V1.0 [조장] 최종 확인 완료 Merge branch 'dev'
|\
| *   94d99e8 (origin/dev, dev) V0.3 Merge pull request #6 from Fullstack-Tests/feature/원병선
| |\
| | * bc002ac 원병선 - 테스트 작성 : "대출 비즈니스 예외: 재고 없음이면 400 + error 메시지"
| | * 656dece 원병선 - 테스트 작성 : "대출 검증 실패: bookId 누락 / member 공백이면 400"
| | * c8969bd 원병선 - 테스트 작성 : "대출 정상: 200 과 lendId 를 반환"
| |/
|/|
| *   e55cf96 V0.2 Merge pull request #5 from Fullstack-Tests/feature/박영준
| |\
| | * 6f99378 박영준 - isOverdue_경계 Test
| | * b18adb7 박영준 - returnBook_중복반납_차단 Test
| | * cea1320 박영준 - lend_미존재도서_예외 Test
| | * cdf81ce 박영준 - lend_재고0_예외 Test
| | * faf4bd7 박영준 - lend_정상_ Test
| |/
|/|
| * b025597 V0.1 Merge pull request #2 from Fullstack-Tests/feature/박한결
|/|
| * dd4b754 박한결 - 재고 조회 API 호출
| * cddcdb8 박한결 - 도서 대출 API 호출
| * 555a706 박한결 - 테스트용 도서 저장
| * 697d814 박한결 - ISBN 존재 여부 검증
| * 3e3f2e5 박한결 - ISBN 조회 검증
| * 33855f7 박한결 - 테스트용 도서 저장
|/
* 0dd9c10 조장 - 베이스 세팅
* b69dceb Initial commit
```

commit 확인

2. BookControllerTest – 원병선

```
@Test  byeongseon8-arch +1
@DisplayName("대출 정상: 200 과 lendId 를 반환한다")
void lend_정상_200() throws Exception {
    // 도서 대출 요청이 유효할 때 컨트롤러가 정상 처리 결과물을 리턴하는지 검증하는 메서드
    Lend lend = Lend.builder().id(10L).dueAt(LocalDateTime.now().plusDays(14)).build();
    // 가짜 서비스가 리턴해 줄 대출 id 10번과 반납기한 14일이라는 정보가 담긴 Lend 결과물을 객체를 조립
    when(libraryService.lend(eq( value: 1L), anyString())).thenReturn(lend);
    // libraryService의 lend 메서드에 책 id 1번과 무작위 회원명이 전달되면,
    // 위에서 만든 lend 객체를 리턴하는 규칙 선언

    mockMvc.perform(post( uriTemplate: "/api/library/lend")
                    // MockMvc를 사용해 "/api/library/lend" 주소로 가상의 HTTP POST 요청을 전송
                    .contentType(MediaType.APPLICATION_JSON)
                    // 서버로 전송하는 포맷이 JSON 문서 구조 임을 헤더에 기입
                    .content("{\"bookId\":1,\"member\":\"kim\"}"))
            // 실제 입력 폼의 데이터 규격(책 번호 1, 회원명 kim)을
            // JSON 텍스트 문자열로 만들어 HTTP Body에 보냄
            .andExpect(status().isOk())
            // 컨트롤러의 응답 결과 상태 코드가 정상 처리를 뜻하는 200 OK 인지 판단
            .andExpect(jsonPath( expression: "$.lendId").value( expectedValue: 10));
    // 응답으로 전달받은 JSON 데이터 중 'lendId'라는 이름의 필드 값 구조가
    // 가짜 서비스에서 넘겨준 id 10과 일치하는지 최종 검증
}
```

테스트 작성 : "대출 정상: 200 과 lendId 를 반환"

메서드를 사용하여 도서 대출 요청이 유효할 때 컨트롤러가 정상 처리 결과물을 리턴하는지 검증하고 가짜 서비스를 리턴해줄 정보를 만들고, 메서드에 id, 무작위 회원명이 전달되면 lend 객체를 리턴한다.

MockMvc를 사용해 가상의 HTTP POST 요청을 전송하고 이 포맷이 JSON 문서 구조임을 헤더에 기입한다.
실제 입력 폼의 데이터 규격을 JSON 텍스트 문자열로 만들어 HTTP Body에 보낸다.
컨트롤러의 응답 결과 상태코드가 200 ok면 정상처리를 의미하며 전달받은 데이터 중 LendId라는 이름의 필드 값 구조가 id 10과 일치하는지 최종 검증한다.

2. BookControllerTest – 원병선

```
@Test  byeongseon8-arch +1
@DisplayName("대출 검증 실패: bookId 누락/member 공백이면 400")
void lend_검증실패_400() throws Exception {
    // 필수 요구값 누락 시 컨트롤러 초입의 @Valid 유효성 검증 장치가 정상 작동하여 입구를 차단하는지 검증하는 메서드
    mockMvc.perform(post( uriTemplate: "/api/library/lend")
                    // MockMvc를 통해 동일한 대출 API 엔드포인트로 가상의 POST 요청을 진행
                    .contentType(MediaType.APPLICATION_JSON)
                    // 전송 데이터 규격을 JSON 포맷으로 세팅
                    .content("{\"bookId\":null,\"member\":\"\"}"))
            // 책 ID는 null로 비우고, 회원명은 빈 공백 문자열("")로 조작하여
            // 유효성 규칙을 무시한 데이터를 보냄
            .andExpect(status().isBadRequest());
    //  컨트롤러의 유효성 검증 가드에 걸려 서비스 계층까지 진입 못하고
    //  즉시 400 Bad Request 에러로 튕겨 나갔는지 검증
}
```

테스트 작성 : "대출 검증 실패: bookId 누락 / member 공백이면 400"

메서드를 사용하여 필수 요구값 누락 시 @Valid 유효성 검증 장치가 정상 작동하여 입구를 차단하는지 검증한다. MockMvc를 통해 동일한 대출 API 엔드포인트로 가상의 POST 요청을 진행하고 전송 데이터 규격을 JSON 포맷으로 세팅한다. 책 ID는 null로 비우고, 회원명은 ""로 조작하여 유효성 규칙을 무시하는 데이터를 보낸다. 컨트롤러의 유효성 검증에 통과하지 못하고 400 Bad Request로 진입하지 못했는지 검증한다.

2. BookControllerTest – 원병선

```
@Test  byeongseon8-arch +1
@DisplayName("대출 비즈니스 예외: 재고 없음이면 400 + error 메시지")
void lend_재고없음_400() throws Exception {
    // 도서 재고 부족 예외 발생 시 전역 컨트롤러 어드바이스 등이 예외를 안전하게 포맷팅 응답을 주는지 검증하는 메서드
    when(libraryService.lend(eq( value: 2L), anyString()))
    // 가짜 서비스와 lend 메서드에 책 번호 2번과 아무 회원명이 주입되어 작동을 시작하면,
            .thenThrow(new BizException("대출 가능한 재고가 없습니다."));
            // 정상 객체를 리턴하지 말고 "재고가 없다"는 사유와 BizException 에러를 강제로 발생

    mockMvc.perform(post( uriTemplate: "/api/library/lend")
                    // 책 번호 2번을 빌리겠다는 HTTP POST 요청 패키지를 컨트롤러에 전달
                    .contentType(MediaType.APPLICATION_JSON)
                    // 컨텍트 타입을 JSON 형식으로 체크
                    .content("{\"bookId\":2,\"member\":\"kim\"}"))
            // 책 ID 2번과 회원명 kim 구조로 JSON 바디를 세팅하여 발송
            .andExpect(status().isBadRequest())
            // 가짜 서비스가 던진 비지니스 예외가 컨트롤러에서 캐치되어
            // 최종 400 Bad Request 응답 코드로 매핑 환원되었는지 확인
            .andExpect(jsonPath( expression: "$.error").exists());
    // 사용자 브라우저에게 실어다 주는 최종 에러 JSON 결과 구조 내에 "error"라는
    // 구체적인 사유 키(key) 필드가 실존하여 생성되어 있는지 점검
}
```

테스트 작성 : "대출 비즈니스 예외: 재고 없음이면 400 + error 메시지"

메서드를 통해 도서 재고 부족 예외 발생 시 전역 컨트롤러 등이 예외를 안전하게 포맷팅 응답을 주는지 검증한다.
가짜 서비스와 lend 메서드에 책 번호 2번과 무작위 회원명이 주입되면 "재고가 없다"는 사유와 BizException 에러를 강제로 발생시킨다.
책을 빌리는 HTTP POST 요청 패키지를 컨트롤러에 전달하고 콘텐츠 타입을 JSON 형식으로 체크한다. 이후 구조를 맞추어 JSON 바디를 세팅하여 발송하고 비즈니스 예외가 컨트롤러에서 캐치되어 최종 400 Bad Request 응답 코드로 매핑 변환되었는지 확인한다.
마지막으로 사용자 브라우저에게 실어다 주는 최종 에러 JSON 결과 구조 내에 "error"라는 구체적인 사유 KEY 필드가 생성되어 있는지 점검한다.

3. BookRepositoryTest – 박한결

```
// === 테스트용 도서 저장 ===
// .builder(): 복잡한 객체를 단계별로 생성할 수 있도록 하는 생성 패턴
bookRepository.save(Book.builder() // Book 생성 & DB에 저장
        .title("리팩터링") // 제목 설정
        .isbn("REPO-1") // ISBN 설정
        .totalCopies(2) // 총 권수 설정
        .availableCopies(2) // 대출 가능 권수 설정
        .build() // Book 객체 생성
);

// === ISBN 조회 검증 ===
// findByIsbn(): ISBN 값으로 도서를 조회해 Optional<Book> 형태로 반환하는 메서드
// isPresent(): Optional 객체 내부에 값이 존재하는지 여부를 확인하는 메서드
// 저장되었는지 확인
assertTrue( // 조건이 True인지 검증
        bookRepository.findByIsbn("REPO-1") // ISBN으로 도서 조회
                .isPresent()              // 조회 결과가 존재하는지 확인
);
// 도서 조회
assertEquals( // 두 값이 같은지 검증
        "리팩터링", // 기대 값 (책 제목)
        bookRepository.findByIsbn("REPO-1") // ISBN으로 도서 조회
                .get()                    // Optional에서 Book 객체 꺼내기
                .getTitle()               // Book의 책 제목 가져오기
);
```

테스트용 도서 저장

builder()로 Book 객체를 생성하고 DB에 저장한다.
각 필드(title, isbn, totalCopies, availableCopies)를 설정한 뒤 save()로 저장한다.

ISBN 조회 검증

findByIsbn()으로 도서를 조회하고 isPresent()로 조회 결과 존재 여부를 확인한 뒤, assertTrue()로 해당 도서가 정상적으로 저장되었는지 검증한다.

도서 정보 검증

조회된 Book에서 getTitle()을 통해 제목을 가져와 assertEquals()로 기대값과 동일한지 확인한다.

3. BookRepositoryTest – 박한결

```
// === ISBN 존재 여부 검증 ===
// existsByIsbn(): 특정 ISBN을 가진 데이터가 존재하는지 여부를 boolean으로 반환하는 메서드
// ISBN 존재 여부 확인
assertTrue( // 조건이 True인지 검증
        bookRepository.existsByIsbn("REPO-1")); // ISBN이 존재하는지 확인
// ISBN 미존재 여부 확인
assertFalse( // 조건이 False인지 검증
        bookRepository.existsByIsbn("NONE-0")); // ISBN이 존재하지 않는지 확인
```

ISBN 존재 여부 검증

existsByIsbn()을 통해 특정 ISBN의 존재 여부를 확인하고 assertTrue()로 존재하는 경우를, assertFalse()로 존재하지 않는 경우를 검증한다.

3. LibraryIntegrationTest – 박한결

```
// == 테스트용 도서 저장 ==
// .builder(): 복잡한 객체를 단계별로 생성할 수 있도록 하는 생성 패턴
Book saved = bookRepository.save(Book.builder()     // Book 생성 & DB에 저장
        .title("통합테스트 도서") // 제목 설정
        .isbn("INT-1")         // ISBN 설정
        .totalCopies(1)        // 총 권수 설정
        .availableCopies(1)    // 대출 가능 권수 설정
        .build());             // Book 객체 생성

// === 도서 대출 API 호출 ===

// Perform(): HTTP요청을 실행하는 메서드
// post(): POST 요청 생성
mockMvc.perform(post( uriTemplate: "/api/library/lend")
                .contentType(MediaType.APPLICATION_JSON) // contentType(): 요청 데이터 타입 지정 (JSON)
                .content("{\"bookId\":" + saved.getId() + ",\"member\":\"hong\"}")) // content(): 요청 Body 지정
        .andExpect(status().isOk()) // 응답 상태가 200(OK)인지 확인
        .andExpect(jsonPath( expression: "$.lendId").exists()); // 응답에 lendId가 존재하는지 확인
```

테스트용 도서 저장

builder()를 통해 Book 객체를 생성하고 각 필드(title, isbn, totalCopies, availableCopies)를 설정한 뒤 save()로 DB에 저장한다.

도서 대출 API 호출

mockMvc.perform()으로 /api/library/lend POST 요청을 보내고, contentType()과 content()으로 JSON 요청 형식과 바디를 전달한다.
status().isOk()로 응답 상태가 정상인지 여부를 확인하고, jsonPath("$.lendId").exists()로 대출 ID가 존재하는지 검증한다.

4. LendServiceTest- 박영준

```java
@Test  pyj-0313 +1
@DisplayName("대출 정상: 재고가 1 줄고 반납기한이 설정된다")
void lend_정상() {
    // 테스트용 도서 객체를 생성하고 데이터를 넣는다
    Book book = Book.builder().id(1L).title("클린 코드").isbn("i-1").totalCopies(3).availableCopies(2).build();
    // findById가 호출되면 1L에 저장된 테스트용 도서 데이터를 꺼낸다
    when(bookRepository.findById(1L)).thenReturn(Optional.of(book));
    // lendRepository.save가 어떤 객체로 호출되든 넘겨받은 lend 객체(첫 번째 인자)를 그대로 리턴하도록 설정
    when(lendRepository.save(any(Lend.class))).thenAnswer( InvocationOnMock inv -> inv.getArgument( index: 0));
    // 1번 도서를 "kim"이 대출하도록 lend() 메서드 실행
    Lend lend = libraryService.lend( bookId: 1L,  member: "kim");

    assertEquals( expected: 1, book.getAvailableCopies()); // 책의 재고가 1이 맞는지 확인
    assertNotNull(lend.getDueAt()); // dueAt(반납기한)이 null이면 안된다
    assertEquals( expected: "kim", lend.getMember()); // 책을 대출한 사람이 "kim"이 맞는지 확인
}
```

```java
@Test  pyj-0313 +1
@DisplayName("대출 경계: 재고 0 이면 대출할 수 없다 (D1 경계값)")
void lend_재고0_예외() {
    // 테스트용 도서 객체를 생성하고 데이터를 넣는다
    Book book = Book.builder().id(1L).availableCopies(0).build();
    // findById가 호출되면 1L에 저장된 테스트용 도서 데이터를 꺼낸다
    when(bookRepository.findById(1L)).thenReturn(Optional.of(book));
    // 재고가 0인 도서를 대출하려고 하면 BizException이 발생해야 함
    assertThrows(BizException.class, () -> libraryService.lend( bookId: 1L,  member: "kim"));
}
```

도서 대출 테스트

builder()을 통해 재고 2개짜리 도서 객체를 생성하고, findById(1L) 호출 시 반환되도록 설정한다. save() 호출시에는 넘겨받은 Lend 객체를 그대로 반환하도록 설정한다. lend(1L, "kim")을 실행한 뒤, 도서의 availableCopies가 1로 감소했는지, dueAt이 null이 아닌지, member가 "kim"으로 저장되었는지 검증한다.

도서 대출불가 테스트

builder()를 통해 재고가 0인 도서 객체를 생성하고, findById(1L) 호출 시 이 도서가 반환되도록 설정한다. 이 상태에서 lend(1L, "kim")을 실행하면 BizException이 발생하는지 assertThrows로 검증한다.

4. LendServiceTest- 박영준

```java
@Test  pyj-0313 +1
@DisplayName("대출 예외: 존재하지 않는 도서면 BizException (D2)")
void lend_미존재도서_예외() {
    // findById(99L)이 호출되면 해당 id의 도서가 존재하지 않는 상황을 리턴하도록 설정
    when(bookRepository.findById(99L)).thenReturn( value: Optional.empty());
    // 존재하지 않는 도서를 대출하려고 하면 BizException이 발생해야 함
    assertThrows(BizException.class, () -> libraryService.lend( bookId: 99L,  member: "kim"));
}
```

```java
@Test  pyj-0313 +1
@DisplayName("반납 예외: 두 번 반납해도 재고는 한 번만 늘고 두 번째는 BizException (D3 중복 반납)")
void returnBook_중복반납_차단() {
    // 테스트용 도서 객체를 생성하고 데이터를 넣는다
    Book book = Book.builder().id(1L).availableCopies(1).build();
    // 테스트용 대출 기록을 생성하고 데이터를 넣는다
    Lend lend = Lend.builder().id(1L).bookId(1L).build();
    // findById가 호출되면 1L에 저장된 테스트용 도서대출 데이터를 꺼낸다
    when(lendRepository.findById(1L)).thenReturn(Optional.of(lend));
    // findById가 호출되면 1L에 저장된 테스트용 도서 데이터를 꺼낸다
    when(bookRepository.findById(1L)).thenReturn(Optional.of(book));
    // lendRepository.save가 어떤 객체로 호출되든 넘겨받은 lend 객체(첫 번째 인자)를 그대로 리턴하도록 설정
    when(lendRepository.save(any(Lend.class))).thenAnswer( InvocationOnMock inv -> inv.getArgument( index: 0));

    // 첫 번째 반납 : 정상 처리, 재고 1 -> 2로 증가
    libraryService.returnBook( lendId: 1L);
    // 두 번째 반납 : 같은 객체 이미 반납 처리(returnAt 존재) BizException 발생해야 함
    assertThrows(BizException.class, () -> libraryService.returnBook( lendId: 1L));
    // 도서 재고는 2로 그대로 유지되어야 함
    assertEquals( expected: 2, book.getAvailableCopies());
}
```

미존재 도서 대출 테스트

findById(99L) 호출 시 빈 Optional을 반환하도록 설정한다. 존재하지 않는 도서 id로 lend(99L, "kim")을 실행하면 BizException이 발생하는지 검증한다.

중복반납 차단 테스트

builder()를 통해 재고 1개짜리 도서와 미반납 상태의 대출 기록을 각각 생성하고, findById() 호출 시 각각 반환되도록 설정한다. returnBook(1L)을 첫 번째로 실행하면 정상 처리되어 재고가 2로 증가하는지, 같은 id로 두 번째 실행하면 이미 반납 처리된 상태라 BizException이 발생하는지, 그리고 재고는 2로 그대로 유지되는지 검증한다.

4. LendServiceTest- 박영준

```java
@Test  pyj-0313 +1
@DisplayName("연체 경계: 반납기한 당일은 연체가 아니고 그 이후가 연체다 (D4 경계값)")
void isOverdue_경계() {
    // 반납기한(dueAt)을 2026-01-15 10:00로 설정한 대출 기록을 생성
    LocalDateTime due = LocalDateTime.of( year: 2026,  month: 1,  dayOfMonth: 15,  hour: 10,  minute: 0);
    Lend lend = Lend.builder().dueAt(due).build();

    // 반납기한 하루 전 확인 -> 연체 X
    assertFalse(libraryService.isOverdue(lend, due.minusDays(1)));
    // 반납기한 당일 확인 -> 연체 X
    assertFalse(libraryService.isOverdue(lend, due));
    // 반납기한 1초라도 지남 -> 연체 O
    assertTrue(libraryService.isOverdue(lend, due.plusSeconds(1)));

    // 이미 반납된(returnedAt 존재) 대출 기록 생성
    Lend returned = Lend.builder().dueAt(due).returnedAt(due.minusDays(2)).build();
    // 이미 반납된 대출은 기간 이후 확인 시 연체 처리되어 있으면 안됨
    assertFalse(libraryService.isOverdue(returned, due.plusDays(10)));
}
```

도서연체 테스트

builder()를 통해 반납기한이 설정된 대출 기록을 생성하고, isOverdue()를 여러 시점(기한 하루 전, 기한 당일, 기한 1초 후)에 호출하여 각각 연체 여부가 올바르게 반환되는지 검증한다. 이어서 이미 반납 완료된 대출 기록을 생성하고, 기한이 지난 시점에 호출해도 연체로 판단되지 않는지 검증한다.



감사합니다

Fullstack